

Week 12: Learning from Sequences: Time Series, Forecasting and Recurrent Networks

Artificial Intelligence Applications in Engineering (MUH-920 Mühendislikte Yapay Zeka Uygulamaları). Lecture notes

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 | utkukose.com | github.com/utkukose

utkukose@sdu.edu.tr | utku.kose@und.edu | ukose@up.edu.mx | utkukose@gmail.com

Week overview, study path and tasks: <https://github.com/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-12/README.md>

Lecture page with animations and knowledge checks: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-12/lecture.html>

Interactive lab with self-assessment and reflection: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-12/lab.html>

Colab notebook with Python step 12 and the hands-on work:

https://colab.research.google.com/github/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-12/NB12_time_series_forecasting.ipynb

Content licensed under CC BY 4.0. This course is updated in line with current developments in the field.

Last update: October 2026.

Overview

Engineering data often arrive in order: hourly electricity demand, daily river flows, monthly production, the vibration of a machine over its life. Order carries information and imposes rules: A forecast may use only the past, and validation must respect time [1, 2]. This week decomposes series into trend and seasonality, builds honest baselines, fits regression models with seasonal terms and lag features, and trains a long short-term memory network [3]. The running example is the record of atmospheric carbon dioxide at Mauna Loa, started by Keeling in 1958 and maintained by NOAA and the Scripps Institution of Oceanography [4, 5].

Estimated study time: 10 to 12 hours.

Final project. The final project is released this week and is due at the end of Week 14. It builds a learning system for an engineering problem and counts 60 percent of the course grade. The weekly task of this week includes the project proposal, which can be sent by e-mail for early feedback. The [project page](#) describes the options, the proposal, the deliverables and the evaluation criteria.

Learning outcomes

By the end of the week, students are expected to describe trend, seasonality, autocorrelation and noise in a series, to build naive and seasonal naive baselines, to split data chronologically and evaluate forecasts with rolling origins, to construct lag windows for supervised learning, to fit trend-and-season regression and recursive multi-step forecasts, to train an LSTM in PyTorch, and to compare all models honestly on the same test period.

Python in this week

These notes close with Python step 12: Time-indexed data, baselines and exceptions. It covers dates and time-indexed Series in pandas, resampling, rolling windows and shifts for lagged inputs, baseline forecasts compared on a time-ordered split, and exceptions with try and except, applied to temperature records and messy logger data [6]. In the course materials, the step is taught at the start of the Colab notebook, where each snippet runs in a cell next to its explanation and short exercises give immediate feedback, and later sections of the notebook mark further constructs as Python moves.

Time series in engineering

A time series is a sequence of measurements indexed by time. Its behaviour usually combines a trend, a slow change of level; seasonality, a pattern that repeats with a fixed period such as a day or a year; cycles without a fixed period; and irregular noise. Neighbouring values are correlated, which is the property forecasting exploits and the property that breaks the independence assumption of the random splits used in Weeks 7 and 8. Box and Jenkins built a systematic methodology on autocorrelation and differencing [1], and Hyndman and Athanasopoulos give a modern, practical account of forecasting with open examples [2].

The Keeling curve is a textbook series. Continuous measurements at Mauna Loa since 1958 show the carbon dioxide concentration rising every year, with an annual cycle of a few parts per million caused by the growth and decay of vegetation in the northern hemisphere [4, 5]. The rise itself accelerates, so a straight line underestimates the future.

Check your understanding. Why must the test set of a forecasting problem lie after the training period?

- A. Because random splits are slower to compute
- B. Because a forecast can use only the past; random splits let the model see neighbouring future values
- C. Because test sets must always be small
- D. Because seasonality disappears in random splits

Answer: B. With autocorrelated data, a random split leaks information from the future into training and makes forecasts look better than they are.

Formulas: Components, differences and autocorrelation

A time series $y_1, y_2, \dots, y_T$ is often described as the sum of a trend T_t , a seasonal component S_t that repeats with the period m , for example $m = 12$ for monthly data with a yearly cycle, and a remainder R_t [2]:

$$y_t = T_t + S_t + R_t$$

When the seasonal swings grow with the level, a product $y_t = T_t \times S_t \times R_t$ fits better, and taking logarithms turns it into a sum. Differences remove a trend or a season:

$$\nabla y_t = y_t - y_{t-1}, \nabla_m y_t = y_t - y_{t-m}$$

The autocorrelation at lag k measures how strongly the series is correlated with itself k steps earlier. With the mean $\bar{y}$ of the series [1]:

$$r_k = \frac{\sum_{t=k+1}^T (y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^T (y_t - \bar{y})^2}$$

For a series without any autocorrelation, about 95% of the values r_k lie between $-1.96/\sqrt{T}$ and $1.96/\sqrt{T}$ [2]. A seasonal series has large values of r_k at multiples of the period, and a trending series has values that decrease slowly with k .

Worked example: Autocorrelation of a quarterly series

The electricity demand of a small plant, in MWh per quarter over two years, is 52, 40, 45, 60, 56, 43, 48 and 64.

Step 1. Mean and deviations. The mean is $\bar{y}=408/8=51$. The table lists the deviations $d_t=y_t-51$, their squares and the products of each deviation with the deviation one, two and four quarters earlier:

t	y_t	d_t	d_t^2	$d_t d_{t-1}$	$d_t d_{t-2}$	$d_t d_{t-4}$
1	52	1	1			
2	40	-11	121	-11		
3	45	-6	36	66	-6	
4	60	9	81	-54	-99	
5	56	5	25	45	-30	5
6	43	-8	64	-40	-72	88
7	48	-3	9	24	-15	18
8	64	13	169	-39	-104	117
Sum	408	0	506	-9	-326	228

Step 2. Lag 1. The seven products of neighbouring deviations add up to -9:

$$r_1 = \frac{-9}{506} = -0.017787$$

Step 3. Lag 2. The six products add up to -326:

$$r_2 = \frac{-326}{506} = -0.644269$$

Step 4. Lag 4. The four products add up to 228:

$$r_4 = \frac{228}{506} = 0.450593$$

Step 5. Reading the values. The positive r_4 shows the yearly pattern: Each quarter resembles the same quarter a year earlier. The strongly negative r_2 shows that quarters half a year apart lie on opposite sides of the mean. With only eight values, the band $\pm 1.96/\sqrt{8} = \pm 0.692965$ is wide, so such conclusions need longer records. The seasonal differences $y_t - y_{t-4}$ are 4, 3, 3, 4, nearly constant, which reveals a trend of about 3.5 MWh per year that the season had hidden. Part C of the [interactive lab](#) computes autocorrelations and differences of any series.

Baselines and honest evaluation

Every forecasting study needs baselines. The naive forecast repeats the last observation, and the seasonal naive forecast repeats the value from one season earlier, for example the same month of the previous year [2]. A sophisticated model that cannot beat them has learned nothing useful. Evaluation must respect time: The model is trained on data up to a forecast origin and tested on the following horizon. Rolling-origin evaluation repeats this for several origins and averages the errors, which gives a more reliable picture than a single split. Errors grow with the horizon, so results should be reported per

horizon or for the horizon that matters for the decision, such as one day ahead for unit commitment in a power system [7].

Animation: From a series to training examples. A window of past values forms the input and the next values form the target. Sliding the window through the training period creates the examples on which a model is trained. Change the window and horizon lengths. Run it on the [lecture page](#).

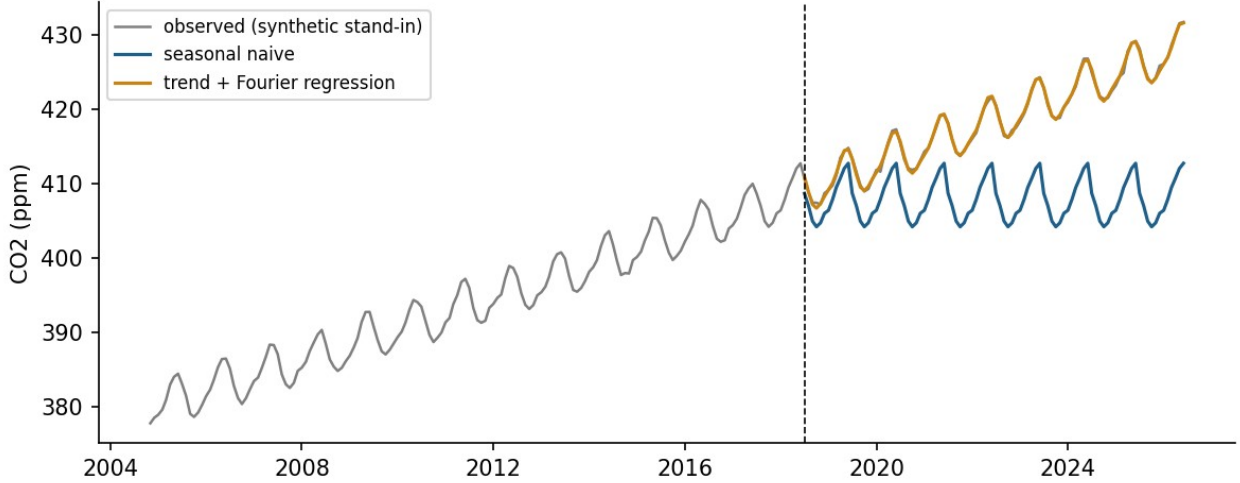


Figure 12.1. Monthly carbon dioxide at Mauna Loa with forecasts for the test period from a seasonal naive baseline and a regression on trend and Fourier terms [5].

Formulas: Baselines and error measures

A forecast made at the origin T for the horizon h is written $\hat{y}_{T+h|T}$. The naive forecast repeats the last observation, the seasonal naive forecast repeats the observation of the same season one period earlier, here for horizons up to one period, and the drift method extends the average change of the past [2]:

$$\hat{y}_{T+h|T} = y_T, \hat{y}_{T+h|T} = y_{T+h-m}, \hat{y}_{T+h|T} = y_T + h \frac{y_T - y_1}{T-1}$$

With the errors $e_t = y_t - \hat{y}_t$ of the n test values, the common error measures are

$$\text{MAE} = \frac{1}{n} \sum |e_t|, \text{RMSE} = \sqrt{\frac{1}{n} \sum e_t^2}, \text{MAPE} = \frac{100}{n} \sum \left| \frac{e_t}{y_t} \right|$$

MAE is in the unit of the data, RMSE weighs large errors more, and MAPE is a percentage that fails when values come close to zero. The mean absolute scaled error divides the MAE by the in-sample MAE of the seasonal naive forecast on the training data, so that values below 1 beat that baseline [2]:

$$\text{MASE} = \frac{\text{MAE}}{\frac{1}{T-m} \sum_{t=m+1}^T |y_t - y_{t-m}|}$$

Rolling-origin evaluation repeats the forecast for several origins $T_1 < T_2 < \dots$ and averages the errors for each horizon.

Worked example: Three baselines and four error measures

The two years of quarterly demand of the previous example form the training data, and the third year, with 59, 47, 51 and 68 MWh, is the test period.

Step 1. Forecasts and errors. The naive forecast repeats the last value, 64. The seasonal naive forecast repeats the four quarters of the second year. A seasonal naive forecast with drift adds the mean yearly change of the training data, the mean of the seasonal differences 4, 3, 3 and 4, which is $14/4 = 3.5$:

Method	Forecasts	Errors $y_t - \hat{y}_t$
Naive	64, 64, 64, 64	-5, -17, -13, 4
Seasonal naive	56, 43, 48, 64	3, 4, 3, 4
Seasonal naive with drift	59.5, 46.5, 51.5, 67.5	-0.5, 0.5, -0.5, 0.5

Step 2. MAE and RMSE.

$$\text{MAE}_{\text{naive}} = \frac{5+17+13+4}{4} = 9.75$$

$$\text{RMSE}_{\text{naive}} = \sqrt{\frac{25+289+169+16}{4}} = \sqrt{124.75} = 11.169154$$

$$\text{MAE}_{\text{seasonal}} = \frac{3+4+3+4}{4} = 3.5, \text{RMSE}_{\text{seasonal}} = \sqrt{\frac{9+16+9+16}{4}} = \sqrt{12.5} = 3.535534$$

and for the forecast with drift every error has the size 0.5, so its MAE and RMSE are both 0.5.

Step 3. MAPE. For the naive forecast, the relative errors are 5/59, 17/47, 13/51 and 4/68:

$$\text{MAPE}_{\text{naive}} = \frac{100}{4} \times (0.084746 + 0.361702 + 0.254902 + 0.058824)$$

$$\text{MAPE}_{\text{naive}} = 25 \times 0.760174 = 19.00435 \%$$

The same computation gives 6.34% for the seasonal naive forecast and 0.9067% with drift.

Step 4. MASE. The in-sample seasonal naive errors of the training data are 4, 3, 3 and 4, with the mean 3.5, so the MASE is $9.75/3.5 = 2.785714$ for the naive forecast, $3.5/3.5 = 1$ for the seasonal naive forecast and $0.5/3.5 = 0.142857$ with drift. Only the forecast that respects both season and trend beats the seasonal naive baseline clearly. Part D of the [interactive lab](#) computes all baselines and measures for any series.

Regression models for time series

Linear regression becomes a forecasting model when its inputs describe time. A polynomial trend captures slow change, and pairs of sine and cosine terms with the seasonal period, called Fourier terms, capture a smooth seasonal pattern with few parameters. Lagged values of the series itself, the value one month ago or twelve months ago, turn forecasting into ordinary supervised learning, so that ridge regression, random forests or gradient boosting can be used. For horizons longer than one step, the model is applied recursively, feeding its own forecasts back as lags, or a separate model is trained for each horizon. Exponential smoothing and ARIMA models are classical alternatives with well understood behaviour and prediction intervals [1, 2].

Check your understanding. A monthly series has a yearly cycle. Which pair of features represents the first harmonic of the season?

- A. month and month squared
- B. $\sin(2 \pi m / 12)$ and $\cos(2 \pi m / 12)$
- C. the value of the previous day
- D. the year only

Answer: B. One sine-cosine pair with period 12 describes a smooth annual cycle with any phase.

Formulas: Trend and Fourier terms, autoregression and exponential smoothing

A regression on time describes the trend by powers of t and a season of period m by pairs of sine and cosine terms, the Fourier terms [2]:

$$y_t = \beta_0 + \beta_1 t + \beta_2 t^2 + \sum_{k=1}^K \left[a_k \sin \frac{2\pi k t}{m} + b_k \cos \frac{2\pi k t}{m} \right] + \varepsilon_t$$

The pair of order $k=1$ describes one smooth wave per period. Since $a \sin \theta + b \cos \theta = A \sin(\theta + \phi)$ with $A = \sqrt{a^2 + b^2}$, the pair has the amplitude A , and its phase sets the season of the peak; higher orders sharpen the shape.

An autoregression predicts a value from the p values before it, which turns forecasting into ordinary regression on lagged inputs [1]:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \dots + \phi_p y_{t-p} + \varepsilon_t$$

For $p=1$, least squares on the pairs $(x_t, y_t) = (y_{t-1}, y_t)$ gives $\phi = S_{xy}/S_{xx}$ and $c = \bar{y} - \phi \bar{x}$, with

$$S_{xy} = \sum x_t y_t - \frac{1}{n} \sum x_t \sum y_t, S_{xx} = \sum x_t^2 - \frac{1}{n} (\sum x_t)^2$$

for the n pairs. Forecasts for several steps use earlier forecasts as inputs, recursively. For $|\phi| < 1$ they approach the mean $\mu = c/(1 - \phi)$ geometrically, since $\hat{y}_{T+h} - \mu = \phi^h (y_T - \mu)$.

Exponential smoothing forms a level ℓ_t as a weighted average of the newest value and the previous level, with a smoothing factor $0 < \alpha \leq 1$; unrolled, the weights of the past values $\alpha, \alpha(1 - \alpha), \alpha(1 - \alpha)^2, \dots$ decay geometrically. Holt's method adds a trend b_t with its own factor β [2]:

$$\ell_t = \alpha y_t + (1 - \alpha)(\ell_{t-1} + b_{t-1}), b_t = \beta(\ell_t - \ell_{t-1}) + (1 - \beta)b_{t-1}$$
$$\hat{y}_{T+h|T} = \ell_T + h b_T$$

Without the trend, $b_t = 0$, this is simple exponential smoothing, whose forecast is the last level for every horizon. The Holt-Winters method adds a third equation for the seasonal component.

Worked example: Fourier terms of a monthly cycle

A model of a monthly carbon dioxide record contains the first Fourier pair with the illustrative coefficients $a = 1.5$ and $b = -2.6$ ppm. Time is counted in months from January, so that the terms with $2\pi t/12$ depend only on the month of the year.

Step 1. The features of May and November. For May, $2\pi \times 5/12$ is 150 degrees, so $\sin = 0.5$ and $\cos = -0.866025$. For November, 330 degrees give $\sin = -0.5$ and $\cos = 0.866025$. The seasonal terms are

$$1.5 \times 0.5 + (-2.6) \times (-0.866025) = 0.75 + 2.251665 = 3.001665$$
$$1.5 \times (-0.5) + (-2.6) \times 0.866025 = -0.75 - 2.251665 = -3.001665$$

Step 2. Amplitude. $A = \sqrt{1.5^2 + 2.6^2} = \sqrt{2.25 + 6.76} = \sqrt{9.01} = 3.001666$ ppm, which agrees with the value of May up to rounding. The seasonal term therefore peaks in May at the value A and reaches its trough, $-A$, six months later in November, as the vegetation of the northern hemisphere takes up carbon dioxide in summer [4].

Step 3. A prediction. With a level of 420 ppm in January and a trend of 0.2 ppm per month, the model predicts for May of the same year $420 + 0.2 \times 4 + 3.001665 = 423.801665$ ppm.

Worked example: An autoregression fitted by hand

A sensor gives the readings 20, 23, 22, 25, 24, 26 and 25. An autoregression of order 1 uses the six pairs (20, 23), (23, 22), (22, 25), (25, 24), (24, 26) and (26, 25).

Step 1. Sums. $\sum x_t = 140$, $\sum y_t = 145$, $\sum x_t^2 = 400 + 529 + 484 + 625 + 576 + 676 = 3290$, and

$$\sum x_t y_t = 460 + 506 + 550 + 600 + 624 + 650 = 3390$$

Step 2. Least squares.

$$S_{xy} = 3390 - \frac{140 \times 145}{6} = 3390 - \frac{20300}{6} = \frac{20}{3}$$

$$S_{xx} = 3290 - \frac{140^2}{6} = 3290 - \frac{19600}{6} = \frac{70}{3}$$

$$\phi = \frac{20/3}{70/3} = \frac{2}{7} = 0.285714, c = \frac{145}{6} - \frac{2}{7} \times \frac{140}{6} = \frac{145 - 40}{6} = 17.5$$

Step 3. Recursive forecasts. From the last reading, 25, and with the exact value $\phi = 2/7$:

$$\hat{y}_{T+1} = 17.5 + \frac{2}{7} \times 25 = 17.5 + 7.142857 = 24.642857$$

$$\hat{y}_{T+2} = 17.5 + \frac{2}{7} \times 24.642857 = 17.5 + 7.040816 = 24.540816$$

$$\hat{y}_{T+3} = 17.5 + \frac{2}{7} \times 24.540816 = 17.5 + 7.011662 = 24.511662$$

Step 4. The long run. The mean is $\mu = 17.5 / (1 - 2/7) = 17.5 / (5/7) = 24.5$. The distance of the forecasts from it shrinks by the factor $2/7$ per step, from $25 - 24.5 = 0.5$ to $0.5 \times 2/7 = 0.142857$, which is $24.642857 - 24.5$. Part E of the [interactive lab](#) fits autoregressions of any series and order.

Worked example: Simple exponential smoothing and Holt's method

The weekly demand for a spare part is 30, 32, 31, 35 and 37 units.

Step 1. Simple exponential smoothing with $\alpha = 0.5$. Starting from $\ell_1 = 30$:

Week	Demand	Level $\ell_t = 0.5 y_t + 0.5 \ell_{t-1}$
2	32	$0.5 \times 32 + 0.5 \times 30 = 31$
3	31	$0.5 \times 31 + 0.5 \times 31 = 31$
4	35	$0.5 \times 35 + 0.5 \times 31 = 33$
5	37	$0.5 \times 37 + 0.5 \times 33 = 35$

The forecast for every following week is 35 units: The method follows the level but not the rise.

Step 2. Holt's method with $\alpha = 0.5$ and $\beta = 0.3$. It starts from $\ell_1 = 30$ and the first change $b_1 = 32 - 30 = 2$. Each week updates the level and then the trend:

$$\ell_2 = 0.5 \times 32 + 0.5 \times (30 + 2) = 32$$

$$b_2 = 0.3 \times (32 - 30) + 0.7 \times 2 = 2$$

$$\ell_3 = 0.5 \times 31 + 0.5 \times (32 + 2) = 32.5$$

$$b_3 = 0.3 \times (32.5 - 32) + 0.7 \times 2 = 1.55$$

$$\ell_4 = 0.5 \times 35 + 0.5 \times (32.5 + 1.55) = 34.525$$

$$b_4 = 0.3 \times (34.525 - 32.5) + 0.7 \times 1.55 = 1.6925$$

$$\ell_5 = 0.5 \times 37 + 0.5 \times (34.525 + 1.6925) = 36.60875$$

$$b_5 = 0.3 \times (36.60875 - 34.525) + 0.7 \times 1.6925 = 1.809875$$

Step 3. Forecasts. With $\ell_5 = 36.60875$ and $b_5 = 1.809875$:

$$\hat{y}_6 = 36.60875 + 1 \times 1.809875 = 38.418625$$

$$\hat{y}_7 = 36.60875 + 2 \times 1.809875 = 40.2285$$

$$\hat{y}_8 = 36.60875 + 3 \times 1.809875 = 42.038375$$

The trend term lets the forecast continue the rise. Part F of the lab runs both methods step by step for any series and factors.

Recurrent neural networks

A recurrent network processes a sequence one step at a time and carries a hidden state from step to step, so that it can summarise the past in a fixed-size vector [8]. Training by backpropagation through time suffers from vanishing and exploding gradients, which makes long-range dependencies hard to learn [9]. The long short-term memory cell adds a memory cell whose input and output gates control what is written into it and read from it, and a forget gate, added in later work, controls what is kept; this design lets information and gradients flow across many steps [3]. The gated recurrent unit is a simpler relative [10]. Convolutional networks over time and attention-based transformers are strong alternatives for sequence modelling [11, 12], and surveys compare these families for forecasting [13].

Deep sequence models need data. A monthly series of about eight hundred values is small for an LSTM, and a well-designed regression with seasonal terms is often as accurate. Recurrent networks show their strength with many related series, such as the load of thousands of feeders, and with long high-frequency records, such as the degradation trajectories of turbofan engines in the C-MAPSS simulation data used for remaining useful life prediction [14].

Check your understanding. What problem does the gating of an LSTM address?

- A. The lack of a bias term in simple networks
- B. The vanishing gradient that prevents simple recurrent networks from learning long-range dependencies
- C. The need for convolution in time series
- D. The calculation of Fourier terms

Answer: B. Gates let information and gradients pass through many steps with little decay [3, 9].

Formulas: The recurrent unit, its gradient and the LSTM cell

A recurrent network reads the inputs $x_1, x_2, \dots$ one at a time and updates a hidden state h_t from the new input and the previous state, with the same weights at every step [8]:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b), \hat{y}_t = W_y h_t + c$$

Backpropagation through time applies the chain rule along the sequence. For a single unit with the weight w_h , the derivative of a later state with respect to an earlier one is a product of one factor per step:

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{j=t-k+1}^t w_h (1 - h_j^2)$$

If the factors are smaller than 1 in size, the product vanishes exponentially with k ; if they are larger, it explodes. Either way, dependencies over many steps are hard to learn [9].

Hochreiter and Schmidhuber introduced a memory cell c_t whose content is changed only by addition, protected by gates [3]. The form used today, for example in PyTorch, has an input gate i_t , a forget gate f_t , an output gate o_t and a candidate g_t , each computed from x_t and h_{t-1} with its own weights:

$$\begin{aligned} i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i), f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \\ g_t &= \tanh(W_g x_t + U_g h_{t-1} + b_g), o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \\ c_t &= f_t c_{t-1} + i_t g_t, h_t = o_t \tanh(c_t) \end{aligned}$$

The derivative of the new cell state with respect to the old one along the direct path is the forget gate itself, $\partial c_t / \partial c_{t-1} = f_t$, so a gate close to 1 carries information and gradients across many steps. An LSTM layer with d inputs and n hidden units has four sets of weights and, in PyTorch, two bias vectors per set:

$$P_{\text{LSTM}} = 4n(d+n) + 8n$$

Worked example: A recurrent unit and its vanishing gradient

A single unit has $w_x = 0.5$, $w_h = 0.8$ and no bias, starts from $h_0 = 0$ and reads the inputs 1, 0.5 and -1:

$$\begin{aligned} h_1 &= \tanh(0.5 \times 1 + 0.8 \times 0) = \tanh(0.5) = 0.462117 \\ h_2 &= \tanh(0.5 \times 0.5 + 0.8 \times 0.462117) = \tanh(0.25 + 0.369694) \\ h_2 &= \tanh(0.619694) = 0.550915 \\ h_3 &= \tanh(0.5 \times (-1) + 0.8 \times 0.550915) = \tanh(-0.5 + 0.440732) \\ h_3 &= \tanh(-0.059268) = -0.059199 \end{aligned}$$

The gradient. The two factors are

$$\begin{aligned} \frac{\partial h_3}{\partial h_2} &= 0.8 \times (1 - 0.003505) = 0.797196 \\ \frac{\partial h_2}{\partial h_1} &= 0.8 \times (1 - 0.303507) = 0.557194 \end{aligned}$$

so $\partial h_3 / \partial h_1 = 0.797196 \times 0.557194 = 0.444193$. Over 20 steps with a typical factor of 0.6, the gradient shrinks to $0.6^{20} = 3.66 \times 10^{-5}$; with a factor of 1.2 it grows to $1.2^{20} = 38.34$.

Worked example: One step of an LSTM cell

A cell with one input and one hidden unit has the state $h_{t-1} = 0.2$ and $c_{t-1} = 0.5$ and receives $x_t = 1$.

Step 1. The gates. Each gate forms its weighted sum $w x_t + u h_{t-1} + b$ with its own weights w , u and bias b , and applies its function:

Gate	Weighted sum	Value
i_t	$0.5 \times 1 + (-0.3) \times 0.2 + 0 = 0.5 - 0.06 + 0 = 0.44$	$\sigma(0.44) = 0.608259$
f_t	$0.8 \times 1 + 0.4 \times 0.2 + 1 = 0.8 + 0.08 + 1 = 1.88$	$\sigma(1.88) = 0.867611$
g_t	$1.2 \times 1 + 0.6 \times 0.2 + (-0.2) = 1.2 + 0.12 - 0.2 = 1.12$	$\tanh(1.12) = 0.807569$
o_t	$0.9 \times 1 + (-0.5) \times 0.2 + 0.1 = 0.9 - 0.1 + 0.1 = 0.9$	$\sigma(0.9) = 0.71095$

Step 2. The new states.

$$c_t = 0.867611 \times 0.5 + 0.608259 \times 0.807569 = 0.433806 + 0.491211 = 0.925017$$

$$h_t = 0.71095 \times \tanh(0.925017) = 0.71095 \times 0.728262 = 0.517758$$

Step 3. The path through the cell. Along the cell state, the gradient is multiplied by the forget gate, 0.867611 here. With forget gates of 0.95, it keeps $0.95^{20} = 0.358486$ of its size over 20 steps, compared with 3.66×10^{-5} for the simple unit.

Step 4. Size of the notebook's model. The LSTM of the notebook has $d = 1$ input and $n = 32$ hidden units, so

$$P_{\text{LSTM}} = 4 \times 32 \times (1 + 32) + 8 \times 32 = 4224 + 256 = 4480$$

and its linear output layer adds $32 + 1 = 33$, for 4513 parameters, which the 24-month windows of the training period must determine. Part G of the [interactive lab](#) runs both cells over a sequence.

Forecasts for decisions

A forecast serves a decision: scheduling generation, sizing a reservoir release, ordering spare parts, or setting emission targets. Point forecasts should therefore come with uncertainty, as prediction intervals or quantiles, and probabilistic forecasting competitions in energy have made this standard practice [7]. The forecast horizon, the update frequency and the cost of errors in each direction belong in the problem statement, as the costs of misses and false alarms did in Week 8.

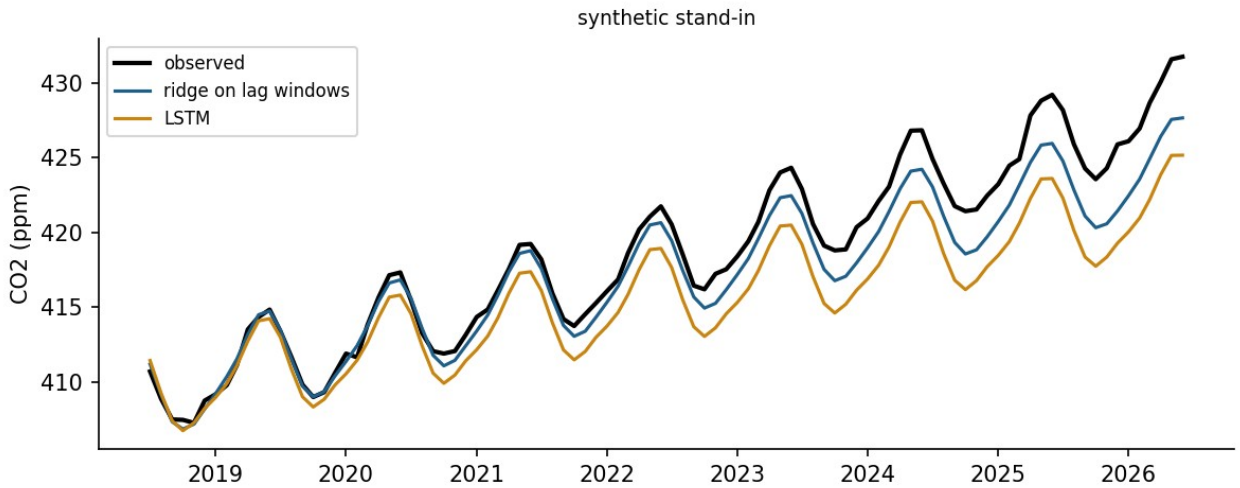


Figure 12.2. Recursive multi-step forecasts of an LSTM and of a ridge regression on lag windows for the same test period, compared with the observations.

Formulas: Prediction intervals, quantiles and costs

If the forecast errors are roughly normal with the standard deviation σ_h at the horizon h , the interval

$$\hat{y}_{T+h|T} \pm z \sigma_h$$

contains the future value with the chosen probability, where $z = 1.96$ for 95% and $z = 1.28$ for 80%. The spread grows with the horizon; for the naive forecast of a series whose changes are independent with the standard deviation σ , it is $\sigma_h = \sigma \sqrt{h}$ [2]. The share of test values that fall inside the intervals, the coverage, should be close to the nominal level.

A quantile forecast q_τ states a value that the observation stays below with the probability τ . It is scored with the pinball loss, which energy forecasting competitions use [7]:

$$L_\tau(y, q) = \tau(y - q) \text{ if } y \geq q, L_\tau(y, q) = (1 - \tau)(q - y) \text{ if } y < q$$

When an underestimate costs c_u per unit and an overestimate c_o per unit, the expected cost of a decision q is $c_u E[\max(0, Y - q)] + c_o E[\max(0, q - Y)]$. Raising q by a small amount saves c_u with the probability $P(Y > q)$ and costs c_o with the probability $P(Y < q)$, so the best decision balances both:

$$c_o P(Y \leq q^*) = c_u (1 - P(Y \leq q^*)), P(Y \leq q^*) = \frac{c_u}{c_u + c_o}$$

The best decision is the quantile of the level $c_u / (c_u + c_o)$ of the forecast distribution, not its mean.

Worked example: Intervals for a load forecast

Step 1. One step ahead. A forecast of tomorrow's peak load is 48 MW, and the one-step errors of the past have the standard deviation 2 MW. The 95% interval is

$$48 \pm 1.96 \times 2 = 48 \pm 3.92, \text{ from } 44.08 \text{ to } 51.92 \text{ MW}$$

Step 2. Four steps ahead. For a naive forecast four days ahead, $\sigma_4 = 2 \times \sqrt{4} = 4$ MW, and the interval widens to $48 \pm 1.96 \times 4 = 48 \pm 7.84$ MW.

Step 3. Coverage. If 18 of 20 test days fall inside their 95% intervals, the coverage is $18/20 = 90\%$. Over many such checks, a coverage persistently below the nominal 95% means that the intervals are too narrow.

Worked example: Pinball loss and the cost of errors

Step 1. Scoring three quantiles. For tomorrow's peak load, the quantile forecasts are $q_{0.1} = 45$, $q_{0.5} = 50$ and $q_{0.9} = 56$ MW, and the observed peak is 53 MW:

$$L_{0.1} = 0.1 \times (53 - 45) = 0.8, L_{0.5} = 0.5 \times (53 - 50) = 1.5$$

$$L_{0.9} = (1 - 0.9) \times (56 - 53) = 0.3$$

with the mean $(0.8 + 1.5 + 0.3) / 3 = 0.866667$. The 90% quantile is penalised lightly because the observation lies below it, as it should with the probability 0.9.

Step 2. The best quantile. A utility buys any shortfall on the balancing market for 60 per MWh more than planned energy, and a surplus loses 20 per MWh, illustrative values. The best plan is the quantile of the level

$$\frac{c_u}{c_u + c_o} = \frac{60}{60 + 20} = 0.75$$

Step 3. The planned load. If the forecast distribution is normal with the mean 50 MW and the standard deviation 4 MW, the 75% quantile is $50 + 0.6745 \times 4 = 52.698$ MW, about 2.7 MW above the mean forecast. Part H of the [interactive lab](#) computes intervals, pinball losses and cost-optimal quantiles.

Python step 12: Time-indexed data, baselines and exceptions

Dates and a time index

Time series carry their time stamps with them. `pd.date_range` creates a sequence of dates, and a Series with such an index can be selected with date strings, grouped by month or resampled to a coarser time step. The practice series below is synthetic: Two years of daily mean temperatures with a mean of 12 °C, a seasonal swing of 11 °C and random variation from day to day.

```
import numpy as np
import pandas as pd

days = pd.date_range("2024-01-01", "2025-12-31", freq="D")
doy = days.dayofyear.to_numpy()
temps = pd.Series(12 + 11 * np.sin(2 * np.pi * (doy - 110) / 365)
                  + np.random.default_rng(12).normal(0, 2.5,
len(days)), index=days, name="temp_C")
print(temps.head(3).round(1))
print(round(float(temps.loc["2025-07"].mean()), 2), "°C mean in July
2025")
```

Output

```
2024-01-01    1.5
2024-01-02    4.1
2024-01-03    3.3
Freq: D, Name: temp_C, dtype: float64
23.09 °C mean in July 2025
```

The selection `temps.loc["2025-07"]` keeps all days of July 2025; a date string may name a year, a month or a single day.

Resampling, rolling windows and lags

`resample("MS").mean()` averages each month and labels it with its first day. `rolling(30, center=True).mean()` smooths the series with a moving window of 30 days, and `shift(k)` moves the series by `k` steps, which creates the lagged inputs used for forecasting. Missing values appear where a shift or a window reaches beyond the data, and `dropna` removes them.

```
monthly = temps.resample("MS").mean()
print(monthly.head(3).round(2))
smooth = temps.rolling(30, center=True).mean()
print(int(smooth.isna().sum()), "days without a full window")
lagged = pd.DataFrame({"today": temps, "yesterday": temps.shift(1),
"last_year": temps.shift(365)}).dropna()
print(lagged.shape)
print(monthly.groupby(monthly.index.month).mean().round(1).head(3))
```

Output

```
2024-01-01    1.25
2024-02-01    2.92
2024-03-01    5.74
Freq: MS, Name: temp_C, dtype: float64
29 days without a full window
(366, 3)
1    1.2
2    2.7
```

```
3      5.5
Name: temp_C, dtype: float64
```

The last line groups the monthly means by calendar month, so January 2024 and January 2025 are averaged together, which is the seasonal profile behind the seasonal baselines of this week.

Baselines on a time-ordered split

A forecast is only useful if it beats simple baselines. For time series the test period must lie after the training period, because shuffled splits would let a model use the future. Three baselines use the year 2024 for training and 2025 for testing. Persistence predicts that tomorrow equals today, the seasonal naive forecast repeats the value 365 days earlier, and the climatology predicts the mean of the same calendar month in the training year.

```
def mae(a, b):
    """Mean absolute error of two aligned Series."""
    return float((a - b).abs().mean())

test = temps.loc["2025"]
train = temps.loc["2024"]
persistence = temps.shift(1).loc["2025"]
seasonal_naive = temps.shift(365).loc["2025"]
profile = train.groupby(train.index.month).mean()
climatology = pd.Series(profile.loc[test.index.month].to_numpy(),
                        index=test.index)
for name, fc in [("persistence", persistence), ("seasonal naive",
seasonal_naive), ("climatology", climatology)]:
    print(f"{name:15s} MAE {mae(test, fc):.2f} °C")
```

Output

```
persistence      MAE 2.65 °C
seasonal naive    MAE 2.93 °C
climatology       MAE 2.15 °C
```

In this synthetic series the variation from day to day is independent noise, so the monthly profile wins. In real temperature records the weather of one day carries over into the next, and persistence becomes a strong competitor for short horizons. Every forecasting model of this week has to beat the best of such baselines on the same test period.

When something goes wrong: exceptions

Real data are messy. A logger may write decimal commas, text such as n/a or empty fields, and converting such text with `float` raises a `ValueError`. A `try` block runs code that may fail, and an `except` block handles the failure instead of stopping the program. The notebooks of this course use the same construction around downloads: If a data service cannot be reached, the `except` block switches to a documented synthetic stand-in. A function can also raise an exception itself when it receives input that makes no sense, which stops errors from spreading silently.

```
raw = ["12.5", "13,1", "n/a", "", "14.0", "-2.3"]
values, failed = [], 0
for text in raw:
    try:
        values.append(float(text.replace(",", ".")))
    except ValueError:
        failed += 1
print(values, "failed:", failed)
```

```
def moving_average(x, window):
    """Mean of every run of `window` consecutive values; window must be
    at least 1."""
    if window < 1:
        raise ValueError("window must be at least 1")
    return [sum(x[i:i + window]) / window for i in range(len(x) -
window + 1)]

print(moving_average(values, 2))
try:
    moving_average(values, 0)
except ValueError as error:
    print("error:", error)
```

Output

```
[12.5, 13.1, 14.0, -2.3] failed: 2
[12.8, 13.55, 5.85]
error: window must be at least 1
```

Check your understanding. `s` is a monthly Series. What does `s.shift(1)` contain in its first position?

- A. The value of the last month
- B. A missing value (NaN)
- C. Zero
- D. The same value as `s` in its first position

Answer: B. Each value moves one step later in time, so nothing is available to fill the first position.

Discipline challenges

Ordered data exist in every field. Choose a row, respect time in every split, and always report the naive baselines next to your model.

Department	Challenge
Environmental Engineering and Earth Sciences Engineering	Carbon dioxide at Mauna Loa, as in the notebook, or local air quality series [5].
Civil Engineering and Environmental Engineering	Daily precipitation and temperature of Isparta from the Open-Meteo archive [15].
Electrical and Electronics Engineering	Hourly electricity demand or photovoltaic output with daily and weekly seasonality [7].
Mechanical Engineering and Automotive Engineering	Remaining useful life from degradation trajectories such as the C-MAPSS simulations [14].
Geophysical Engineering	Daily counts of earthquakes in a region from the USGS catalogue, with the aftershock decay after large events [16].
Industrial Engineering	Monthly demand of a product family and its effect on inventory decisions.
Food Engineering	Temperature records of a cold chain and forecasts of excursions.
Chemical Engineering and Chemistry	Soft sensing: predicting a slow laboratory measurement from fast process signals.

Department	Challenge
Physics and Mathematics	Forecasting a chaotic system such as the logistic map or the Lorenz equations, and the limits of predictability [17].
Statistics	Compare seasonal naive, exponential smoothing and ARIMA with prediction intervals on one series [1].

Weekly task

Forecast a time series from your department or from a public source such as NOAA, Open-Meteo or the USGS. Split the data chronologically, report the seasonal naive baseline, a regression with trend and seasonal terms and one learned model with lag windows or an LSTM, and evaluate them with at least three rolling origins. Discuss in about 500 words how the forecast would be used and how its uncertainty should be communicated, citing at least three works from this week's references [2, 3, 7]. This week also opens the final project: Write its proposal of about 600 words as described on the [final project page](#), and send it by e-mail for early feedback during an active semester.

Research and report assignment (optional)

Forecasting for engineering operations. Review forecasting methods for one engineering application, such as electricity load, renewable generation, river flow, traffic or remaining useful life. Compare statistical, machine learning and deep learning approaches, the evaluation designs, and the treatment of uncertainty [7, 13, 14].

Both assignments are optional. During an active semester, the weekly task can be sent together with the learning log exported from the interactive lab, and the research report on its own, to utkukose@sdu.edu.tr or utkukose@gmail.com. They carry no separate weight, but the instructor may take them into account as a discretionary adjustment of the midterm component of the grade. The assessment section of the syllabus gives the details, and research reports follow the report template of the course repository.

References

- [1] Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time Series Analysis: Forecasting and Control* (5th ed.). Wiley.
- [2] Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/>
- [3] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- [4] Keeling, C. D., Bacastow, R. B., Bainbridge, A. E., Ekdahl, C. A., Guenther, P. R., Waterman, L. S., & Chin, J. F. S. (1976). Atmospheric carbon dioxide variations at Mauna Loa Observatory, Hawaii. *Tellus*, 28(6), 538-551. <https://doi.org/10.1111/j.2153-3490.1976.tb00701.x>
- [5] Lan, X., & Keeling, R. (2026). *Trends in atmospheric carbon dioxide: Mauna Loa CO2 monthly mean data*. NOAA Global Monitoring Laboratory and Scripps Institution of Oceanography. <https://gml.noaa.gov/ccgg/trends/>
- [6] McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56-61). <https://doi.org/10.25080/Majora-92bf1922-00a>
- [7] Hong, T., Pinson, P., Fan, S., Zareipour, H., Troccoli, A., & Hyndman, R. J. (2016). Probabilistic energy forecasting: Global Energy Forecasting Competition 2014 and beyond. *International Journal of Forecasting*, 32(3), 896-913. <https://doi.org/10.1016/j.ijforecast.2016.02.001>

- [8] Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14(2), 179-211. https://doi.org/10.1207/s15516709cog1402_1
- [9] Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2), 157-166. <https://doi.org/10.1109/72.279181>
- [10] Cho, K., van Merriënboer, B., Gulcehre, C., et al. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1724-1734). <https://doi.org/10.3115/v1/D14-1179>
- [11] Bai, S., Kolter, J. Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. arXiv preprint arXiv:1803.01271. <https://arxiv.org/abs/1803.01271>
- [12] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30* (pp. 5998-6008). <https://arxiv.org/abs/1706.03762>
- [13] Lim, B., & Zohren, S. (2021). Time-series forecasting with deep learning: A survey. *Philosophical Transactions of the Royal Society A*, 379(2194), 20200209. <https://doi.org/10.1098/rsta.2020.0209>
- [14] Saxena, A., Goebel, K., Simon, D., & Eklund, N. (2008). Damage propagation modeling for aircraft engine run-to-failure simulation. In *2008 International Conference on Prognostics and Health Management* (pp. 1-9). IEEE. <https://doi.org/10.1109/PHM.2008.4711414>
- [15] Open-Meteo (2026). *Open-Meteo free weather API: Historical Weather API and Elevation API (data licensed under CC BY 4.0)*. <https://open-meteo.com>
- [16] U.S. Geological Survey (2026). *Earthquake Catalog API (FDSN Event Web Service)*. <https://earthquake.usgs.gov/fdsnws/event/1/>
- [17] Kose, U., & Arslan, A. (2017). Forecasting chaotic time series via ANFIS supported by vortex optimization algorithm: Applications on electroencephalogram time series. *Arabian Journal for Science and Engineering*, 42(8), 3103-3114. <https://doi.org/10.1007/s13369-016-2279-z>